

自然语言处理实验一实验报告

基于不同文本表示的新闻文本分类

姓名：李冉婧 学号：2310511 专业：信息安全

一、实验目的

- 理解不同文本表示方法的基本思想；
- 掌握文本分类任务的基本实验流程；
- 熟悉 Logistic Regression 在文本分类中的使用；
- 掌握 Word2Vec、GloVe 等词向量的基本使用方法；
- 熟悉 BERT 类预训练语言模型的 Fine-tuning 流程；
- 能够使用 Accuracy 和 Macro-F1 对文本分类模型进行评价；
- 能够对不同文本表示方法的性能进行基本比较和分析。

二、实验环境

本实验在 Windows 11 下使用 Python 3.13 虚拟环境完成，主要依赖 scikit-learn、gensim、PyTorch 和 Transformers。具体环境如下：

组件	版本
操作系统	Windows 11
GPU	NVIDIA GeForce RTX 4060 Laptop GPU (8GB)
CUDA	12.8
Python	3.13.9 (Anaconda base, venv)
NumPy	2.5.3
pandas	3.0.6
SciPy	1.18.1
scikit-learn	1.9.1
NLTK	3.10.3
gensim	4.4.0
PyTorch	2.11.0+cu128
Transformers	5.17.0

数据集为 submission/data/nyt.csv (11,519 条, 3 类) 和 submission/data/ag.csv (90,000 条, 仅用于训练 Word2Vec)。

为隔离不同项目的依赖并保证实验可复现，本实验使用独立的 Python 虚拟环境 submission/.venv-bert。所有实验代码均通过该虚拟环境中的解释器运行，不直接使用 Anaconda 基础环境，从而避免与全局环境中的 scipy、scikit-learn 等包发生版本冲突。

```
PS D:\作业\自然语言处理\实验1\submission> .\venv-bert\Scripts\Activate.ps1
```

进入虚拟环境

三、知识背景

3.1 文本分类的基本流程

文本分类通常分为以下步骤：

原始文本

- 分词与规范化
- 文本表示（特征提取）
- 训练分类器
- 在验证集上检查或选择模型
- 在测试集上计算 Accuracy 和 Macro-F1

核心原则是：

- 词表、词频等统计量只能从 Training Set 获得；
- Validation Set 用于监控或选择模型；
- Test Set 只做最终评价，不能反复调参；
- 所有方法使用同一份数据划分，保证可比性。

3.2 分词与词表

BoW 和 Word2Vec 使用统一分词规则：

1. 文本转为小写；
2. 使用 NLTK TreebankWordTokenizer 分词；
3. 删除不含字母或数字的纯标点 token；
4. 保留数字及混合 token。

词表是从训练集统计得到的所有不同词组成的集合：

$$V = \{w_1, w_2, \dots, w_{|V|}\}$$

其中 $|V|$ 表示词表大小。

3.3 Bag-of-Words 原理

Bag-of-Words 把一篇文档表示为一个 $|V|$ 维向量，并忽略词序。

Binary Bag-of-Words 只记录词是否出现：

$$x_i = \begin{cases} 1, & w_i \text{ 在文档中出现} \\ 0, & w_i \text{ 未出现} \end{cases}$$

Word Frequency 记录词的出现次数：

$$x_i = \text{词 } w_i \text{ 在当前文档中的出现次数}$$

BoW 的优点是简单、可解释、训练快；缺点是维度高且稀疏，无法表示词序和语义相似关系。

3.4 Logistic Regression 原理

Logistic Regression 用于多分类时，对类别 c 计算线性得分：

$$z_c = w_c^T x + b_c$$

再通过 Softmax 把得分转为概率：

$$P(y = c | x) = \frac{e^{z_c}}{\sum_{k=1}^K e^{z_k}}$$

训练时最小化交叉熵损失：

$$L = -\frac{1}{N} \sum_{i=1}^N \log P(y_i | x_i)$$

本实验中，所有 Logistic Regression 使用相同配置：C=1.0、solver="lbfgs"、max_iter=2000、random_state=42。

3.5 Word2Vec 原理

Word2Vec 学习每个词的稠密低维向量，其基本假设是：出现在相似上下文中的词往往具有相似含义。

CBOW 根据上下文词预测中心词，目标是最大化：

$$\sum_t \log P(w_t | w_{t-c}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+c})$$

Skip-gram 根据中心词预测上下文词，目标是最大化：

$$\sum_t \sum_{-c \leq j \leq c, j \neq 0} \log P(w_{t+j} | w_t)$$

本实验使用 Skip-gram 结构。训练得到的词向量可用于表示文档。

3.6 GloVe 原理

GloVe 利用全局词共现信息学习词向量。它先统计词 w_i 和 w_j 共同出现的次数 X_{ij} ，然后让词向量内积近似表示共现的对数：

$$w_i^T w_j + b_i + \bar{b}_j \approx \log X_{ij}$$

本实验直接使用公开的 glove.6B.100d 预训练词向量。

3.7 文档向量的平均池化

对于一篇包含 n 个有效词的文档，文档向量通过对所有有效词向量求平均得到：

$$e(d) = \frac{1}{n} \sum_{i=1}^n e(w_i)$$

如果某个词不在词向量表中，则视为 OOV 词并忽略。若一篇文档没有任何有效词，则使用零向量。

3.8 BERT 原理

BERT 基于 Transformer 的自注意力机制，能够根据上下文动态表示每个词。自注意力中，词与词之间通过注意力权重交互：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

BERT 使用 [CLS] token 的表示接入分类层。本实验使用 `google-bert/bert-base-uncased`，在分类任务上端到端微调：

[CLS] 表示 $\rightarrow$ Dropout $\rightarrow$ Linear $\rightarrow$ Softmax $\rightarrow$ 3 类概率

3.9 评价指标

Accuracy 为预测正确样本占全部测试样本的比例：

$$ACC = \frac{\text{预测正确的样本数}}{\text{测试样本总数}}$$

对每个类别分别计算 Precision、Recall 和 F1：

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}, \quad F1 = \frac{2PR}{P + R}$$

Macro-F1 对所有类别的 F1 取平均：

$$Macro-F1 = \frac{1}{K} \sum_{c=1}^K F1_c$$

本实验数据中 sports 约占 75%，因此 Accuracy 容易偏高，必须同时使用 Macro-F1 观察少数类表现。

四、实验内容与步骤

4.1 数据与划分

NYT 数据包含 text 和 label 两列，类别为 business、politics、sports：

类别	样本数	占比
business	1,429	12.40%
politics	1,451	12.60%
sports	8,639	74.99%

数据采用分层随机划分：

```
train, temp = train_test_split(
    df, test_size=0.20, random_state=42, stratify=df["label"]
)
val, test = train_test_split(
    temp, test_size=0.50, random_state=42, stratify=temp["label"]
)
```

得到：

类别	Training	Validation	Test
business	1,143	143	143
politics	1,161	145	145
sports	6,911	864	864

所有任务都读取同一份划分，保证比较公平。

4.2 统一预处理

BoW 和 Word2Vec 共用分词函数：

```
def tokenize_text(text: str) -> list[str]:
    tokens = _TOKENIZER.tokenize(str(text).lower())
    return [token for token in tokens if any(ch.isalnum() for ch in token)]
```

BERT 使用其自身的 WordPiece Tokenizer，不能复用上述分词结果，因为 Transformer 模型必须使用与预训练模型匹配的分词器。

4.3 Task 1: Bag-of-Words

- 1. 在 Training Set 上构建词表，得到 142,077 个词；
- 2. 分别生成 Binary BoW 和 Word Frequency 稀疏矩阵；
- 3. 使用 Logistic Regression 训练；
- 4. 在 Test Set 上计算 Accuracy 和 Macro-F1。

运行命令：

```
.\.venv-bert\Scripts\python.exe src\task1_bow.py
```

4.4 Task 2: Word2Vec / GloVe

三种表示统一使用 100 维词向量：

- 1. 加载 GloVe 100d；
- 2. 使用 AG News 90,000 条文本训练 Word2Vec；
- 3. 使用 NYT Training Set 9,215 条文本训练 Word2Vec；
- 4. 对每篇文档取有效词向量的平均；
- 5. 使用 Logistic Regression 训练。

Word2Vec 参数为：

参数	值
vector_size	100
window	5
min_count	5
结构	Skip-gram
negative	5
epochs	5
seed	42

运行命令：

```
.\.venv-bert\Scripts\python.exe src\task2_word2vec.py
```

4.5 Task 3: BERT 微调

项目	配置
模型	google-bert/bert-base-uncased
最大序列长度	64
训练轮数	3 epochs
Batch size	32
优化器	AdamW
学习率	2e-5
Weight decay	0.01

训练 3 个 epoch，每个 epoch 后在 Validation Set 上计算 Macro-F1，最终选择验证集 Macro-F1 最高的模型在 Test Set 上评价。

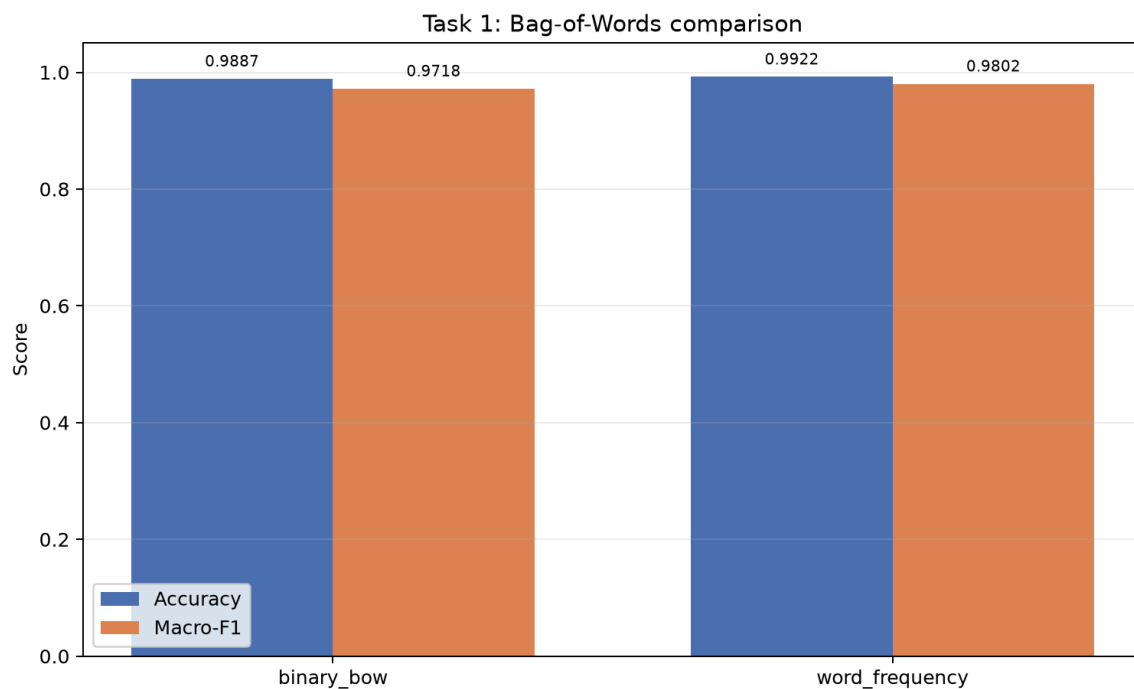
运行命令：

```
.\.venv-bert\Scripts\python.exe src\task3_bert.py --model-name cache\bert-base-uncased-local
```

五、实验结果与分析

5.1 Task 1 结果

表示方法	Validation Accuracy	Validation Macro-F1	Test Accuracy	Test Macro-F1	错分数
Binary BoW	0.9818	0.9562	0.9887	0.9718	13
Word Frequency	0.9809	0.9576	0.9922	0.9802	9



Task 1 对比

```
(.venv-bert) PS D:\作业\自然语言处理\实验1\submission> python src\task1_bow.py
Running binary_bow ...
binary_bow: accuracy=0.9887, macro_f1=0.9718, vocab=142077
Running word_frequency ...
word_frequency: accuracy=0.9922, macro_f1=0.9802, vocab=142077
Task 1 completed in 25.78 seconds.
```

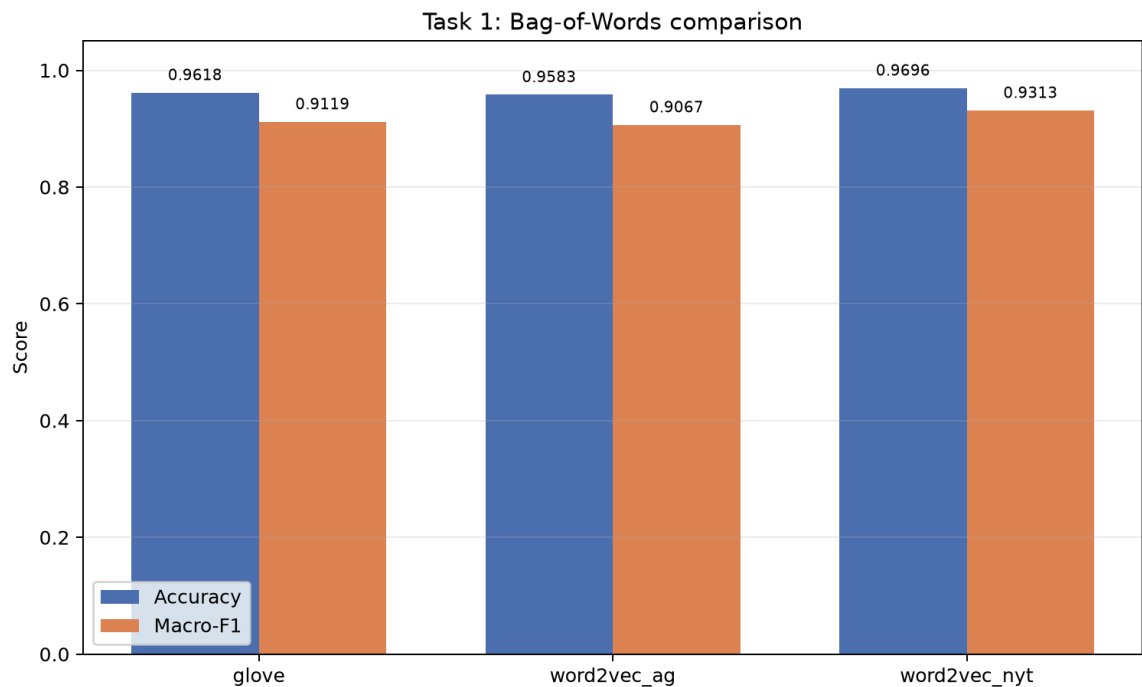
截图1: Task 1 运行结果

Word Frequency 的 Test Accuracy 比 Binary BoW 高约 0.35 个百分点, Macro-F1 高约 0.83 个百分点, 错误数由 13 条减少到 9 条。保留词频后, 模型能利用重复出现的关键词增强类别判断, 对 business 类改善最明显。

两种方法的错误主要发生在 business 和 politics 之间。Logistic Regression 的高权重词符合新闻直觉: business 常见 company、market、bank, politics 常见 washington、republican、obama, sports 常见 game、team、league。

5.2 Task 2 结果

表示方法	Test Accuracy	Test Macro-F1	错分数
GloVe 6B 100d	0.9618	0.9119	44
Word2Vec on AG News	0.9583	0.9067	48
Word2Vec on NYT	0.9696	0.9313	35



Task 2 对比

```
(.venv-bert) PS D:\作业\自然语言处理\实验1\submission> python src\task2_word2vec.py
2026-09-28 07:31:32,418 INFO | Loading and tokenizing NYT
2026-09-28 07:31:46,517 INFO | Loading and tokenizing AG News
2026-09-28 07:31:54,166 INFO | Loading glove vectors from D:\作业\自然语言处理\实验1\submission\cache\glove\glove.6B.100d.txt
2026-09-28 07:31:54,167 INFO | Loading projection weights from D:\作业\自然语言处理\实验1\submission\cache\glove\glove.6B.100d.txt
2026-09-28 07:32:28,997 INFO | KeyedVectors lifecycle event {'msg': 'loaded (400000, 100) matrix of type float32 from D:\作业\自然语言处理\实验1\submission\cache\glove\glove.6B.100d.txt', 'binary': False, 'encoding': 'utf8', 'datetime': '2026-09-28T07:32:28.957742', 'gensim': '4.4.0', 'python': '3.13.9 | packaged by Anaconda, Inc. | (main, Oct 21 2025, 19:09:58) [MSC v.1929 64 bit (AMD64)]', 'platform': 'Windows-11-10.0.26200-SP0', 'event': 'load word2vec format'}
2026-09-28 07:32:28,999 INFO | Loading cached Word2Vec model: D:\作业\自然语言处理\实验1\submission\outputs\task2\models\word2vec_ag.model
2026-09-28 07:32:29,000 INFO | loading Word2Vec object from D:\作业\自然语言处理\实验1\submission\outputs\task2\models\word2vec_ag.model
2026-09-28 07:32:29,026 INFO | loading vw recursively from D:\作业\自然语言处理\实验1\submission\outputs\task2\models\word2vec_ag.model.vw.* with mmap=None
2026-09-28 07:32:29,026 INFO | setting ignored attribute cum_table to None
2026-09-28 07:32:29,171 INFO | Word2Vec lifecycle event {'fname': 'D:\作业\自然语言处理\实验1\submission\outputs\task2\models\word2vec_ag.model', 'datetime': '2026-09-28T07:32:29.171169', 'gensim': '4.4.0', 'python': '3.13.9 | packaged by Anaconda, Inc. | (main, Oct 21 2025, 19:09:58) [MSC v.1929 64 bit (AMD64)]', 'platform': 'Windows-11-10.0.26200-SP0', 'event': 'loaded'}
2026-09-28 07:32:29,173 INFO | Loading cached Word2Vec model: D:\作业\自然语言处理\实验1\submission\outputs\task2\models\word2vec_nyt.model
2026-09-28 07:32:29,173 INFO | loading Word2Vec object from D:\作业\自然语言处理\实验1\submission\outputs\task2\models\word2vec_nyt.model
2026-09-28 07:32:29,197 INFO | loading vw recursively from D:\作业\自然语言处理\实验1\submission\outputs\task2\models\word2vec_nyt.model.vw.* with mmap=None
2026-09-28 07:32:29,197 INFO | setting ignored attribute cum_table to None
2026-09-28 07:32:29,390 INFO | Word2Vec lifecycle event {'fname': 'D:\作业\自然语言处理\实验1\submission\outputs\task2\models\word2vec_nyt.model', 'datetime': '2026-09-28T07:32:29.390518', 'gensim': '4.4.0', 'python': '3.13.9 | packaged by Anaconda, Inc. | (main, Oct 21 2025, 19:09:58) [MSC v.1929 64 bit (AMD64)]', 'platform': 'Windows-11-10.0.26200-SP0', 'event': 'loaded'}
2026-09-28 07:32:29,391 INFO | Building document vectors for glove
2026-09-28 07:32:57,094 INFO | glove coverage: train=0.9543 validation=0.9540 test=0.9540
2026-09-28 07:32:57,354 INFO | glove: accuracy=0.9618 macro_f1=0.9119 errors=44
2026-09-28 07:32:57,356 INFO | Building document vectors for word2vec_ag
2026-09-28 07:33:25,143 INFO | word2vec_ag coverage: train=0.9278 validation=0.9266 test=0.9276
2026-09-28 07:33:25,395 INFO | word2vec_ag: accuracy=0.9583 macro_f1=0.9067 errors=48
2026-09-28 07:33:25,397 INFO | Building document vectors for word2vec_nyt
2026-09-28 07:33:53,246 INFO | word2vec_nyt coverage: train=0.9738 validation=0.9684 test=0.9686
2026-09-28 07:33:53,482 INFO | word2vec_nyt: accuracy=0.9696 macro_f1=0.9313 errors=35
2026-09-28 07:33:53,631 INFO | Task 2 completed in 141.21 seconds
```

截图2: Task 2 运行结果

5.2.1 OOV 与覆盖率

表示方法	词表大小	Test token 覆盖率	Test OOV 类型率	平均有效词数
GloVe	400,000	95.40%	36.58%	628.52
Word2Vec on AG News	26,072	92.76%	63.59%	611.08
Word2Vec on NYT	34,765	96.86%	43.74%	638.11

NYT Word2Vec 的训练语料与测试任务同域，因此测试 token 覆盖率最高，分类效果也最好。AG News Word2Vec 词表最小、OOV 类型率最高，是三种词向量中表现最弱的原因之一。GloVe 词表覆盖广，优于 AG News Word2Vec，但领域适配程度不如 NYT Word2Vec。

5.2.2 与 BoW 的比较

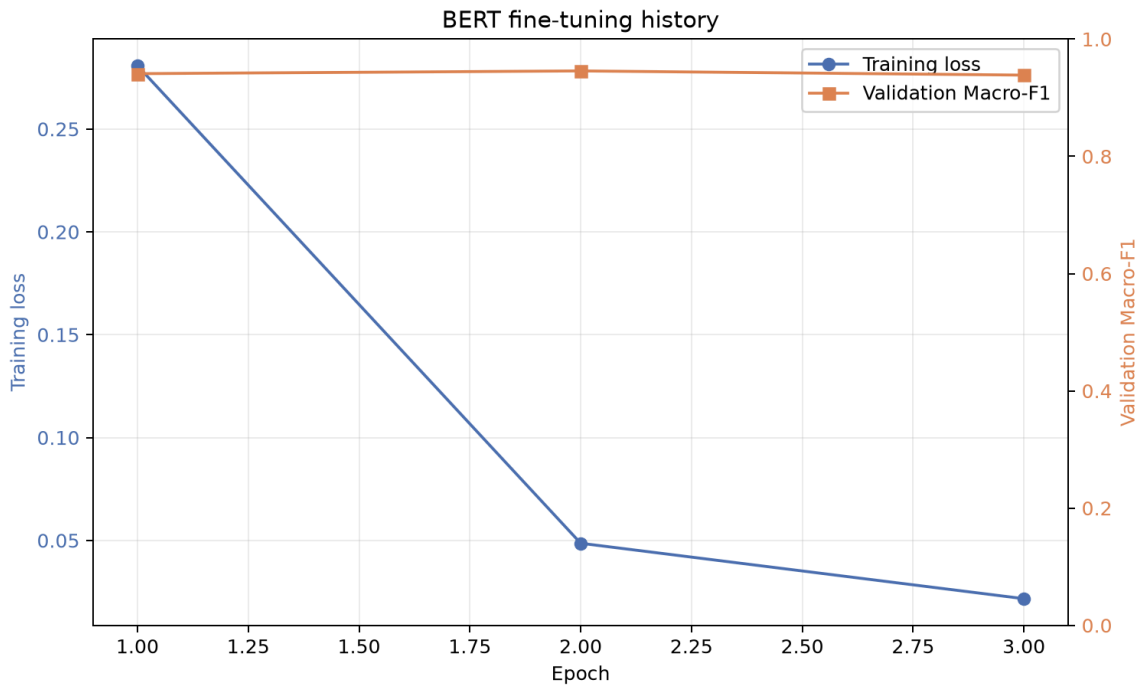
三种词向量的 Macro-F1 都低于 Task 1 的 Bag-of-Words。原因是平均池化把整篇文档压缩为 100 维向量，丢失词序和关键词强度；长新闻中大量背景内容会稀释分类特征。而高维稀疏 BoW 能直接利用具体关键词，在这类类别词汇差异明显的任务中仍然有效。

5.3 Task 3 结果

指标	值
Test Accuracy	0.9852
Test Macro-F1	0.9684
最佳 epoch	2
最佳 Validation Macro-F1	0.9457
错分样本数	17

逐轮训练记录：

Epoch	Train Loss	Validation Loss	Validation Accuracy	Validation Macro-F1
1	0.2807	0.0819	0.9731	0.9411
2	0.0486	0.0835	0.9748	0.9457
3	0.0217	0.0986	0.9714	0.9386



BERT 训练曲线

```
(.venv-bert) PS D:\作业\自然语言处理\实验1\submission> python src\task3_bert.py --model-name cache\bert-base-uncased-local
2026-09-28 07:34:52,148 | INFO | Using device: cuda
2026-09-28 07:34:52,149 | INFO | Loading tokenizer and model: cache\bert-base-uncased-local
Loading weights: 100%|
[transformers] BertForSequenceClassification LOAD REPORT from: cache\bert-base-uncased-local
Key | Status
-----|-----
cls.seq_relationship.bias | UNEXPECTED
cls.predictions.transform.dense.weight | UNEXPECTED
cls.seq_relationship.weight | UNEXPECTED
cls.predictions.transform.LayerNorm.bias | UNEXPECTED
cls.predictions.transform.dense.bias | UNEXPECTED
cls.predictions.bias | UNEXPECTED
cls.predictions.transform.LayerNorm.weight | UNEXPECTED
classifier.bias | MISSING
classifier.weight | MISSING

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
2026-09-28 07:34:54,022 | INFO | Tokenizing datasets with max length=64
D:\作业\自然语言处理\实验1\submission\venv-bert\Lib\site-packages\torch\autograd\graph.py:869: UserWarning: Memory Efficient attention defaults to a non-deterministic algorithm. To explicitly enable determinism call torch.use_deterministic_algorithms(True, warn_only=False). (Triggered internally at C:\actions-runner\work\pytorch\pytorch\aten\src\ATen\native\transformers\cuda\attention_backward.cu:902.)
  return Variable._execution_engine.run_backward( # calls into the C++ engine to run the backward pass
2026-09-28 07:35:18,860 | INFO | Epoch 1 step 100/288 loss=0.6227
2026-09-28 07:35:41,393 | INFO | Epoch 1 step 200/288 loss=0.3664
2026-09-28 07:36:03,827 | INFO | Epoch 1 complete: train_loss=0.2807 val_loss=0.0819 val_accuracy=0.9731 val_macro_f1=0.9411
2026-09-28 07:36:27,196 | INFO | Epoch 2 step 100/288 loss=0.0448
2026-09-28 07:36:50,157 | INFO | Epoch 2 step 200/288 loss=0.0473
2026-09-28 07:37:12,314 | INFO | Epoch 2 complete: train_loss=0.0486 val_loss=0.0835 val_accuracy=0.9748 val_macro_f1=0.9457
2026-09-28 07:37:35,025 | INFO | Epoch 3 step 100/288 loss=0.0144
2026-09-28 07:37:58,719 | INFO | Epoch 3 step 200/288 loss=0.0183
2026-09-28 07:38:20,928 | INFO | Epoch 3 complete: train_loss=0.0217 val_loss=0.0986 val_accuracy=0.9714 val_macro_f1=0.9386
2026-09-28 07:38:28,952 | INFO | Task 3 complete: best epoch=2 accuracy=0.9852 macro_f1=0.9684 errors=17
```

截图3：Task 3 运行结果

分类别结果：

类别	Precision	Recall	F1-score	Support
business	0.9643	0.9441	0.9541	143
politics	0.9459	0.9655	0.9556	145
sports	0.9954	0.9954	0.9954	864

BERT 相比三种平均词向量明显提升了少数类表现。但由于 max_length=64，几乎所有文档都被截断：

数据集	平均 token 数	超过 64 token 的比例
Training	833.44	99.95%
Validation	831.13	100.00%
Test	851.56	100.00%

因此 BERT 只能看到新闻导语，而 BoW 可以看到全文。这解释了为什么本实验中 BERT 略低于 Word Frequency 词袋。

5.4 六种方法最终对比

Task	方法	Test Accuracy	Test Macro-F1
1	Binary BoW	0.9887	0.9718
1	Word Frequency	0.9922	0.9802
2	GloVe 6B 100d	0.9618	0.9119
2	Word2Vec on AG News	0.9583	0.9067
2	Word2Vec on NYT	0.9696	0.9313
3	BERT-base-uncased	0.9852	0.9684

六种方法的混淆矩阵保存在 outputs/task1、outputs/task2、outputs/task3 对应目录中。

六、实验总结

本次实验完成了三种文本表示方法在 NYT 新闻三分类任务上的比较。

在 Bag-of-Words 部分，Binary BoW 和 Word Frequency 都取得了很高效果，其中 Word Frequency 因保留词频信息而优于 Binary BoW，说明在新闻分类中关键词强度是有用的判别信号。

在 Word2Vec 部分，NYT 领域内训练的 Word2Vec 优于 GloVe 和 AG News Word2Vec。领域一致性越高、OOV 越少，平均文档向量越稳定。同时发现，平均词向量在这类长文本、类别词特征明显的任务中不如高维稀疏词袋。

在 BERT 部分，上下文表示和端到端微调使少数类表现明显优于平均词向量方法。但由于题目固定 `max_length=64`，模型只能看到文档开头，最终 Macro-F1 略低于 Word Frequency。

整体而言，本实验最重要的收获是：文本表示方法没有绝对的优劣，需要结合任务特点、文本长度和输入限制来分析。当类别关键词特征明显时，简单词袋可能更强；当需要上下文语义时，BERT 更有优势，但其效果也受到截断长度的约束。

七、复现方式

在 `submission` 目录运行：

```
.\run_all.ps1
```

脚本会自动准备环境、下载 GloVe 和 BERT 权重，并依次运行三个任务。BERT 权重使用 ModelScope 上 `AI-ModelScope/bert-base-uncased` 仓库下载，该仓库与 `google-bert/bert-base-uncased` 是同一模型。